

Arrays and Vectors

CSC 240 Data Structures Fall 2025

Marcus Birkenkrahe

October 30, 2025

Contents

1	Objectives	3
2	Simple example: Array declaration	3
3	Two types of initialization	4
4	PRACTICE Extended example: Declare and Display Product Prices	5
5	Inputting array content element-wise	10
6	Stepping through an array	12
7	PRACTICE Running C++ on the command-line (CLI)	13
8	Initializing a string array	13
9	Partial array initialization	15
10	Reading a file into an array	16
11	Writing an array to a file	18
12	Checkpoint (7.3 No Bounds Checking)	19
13	7.4 The Range-based for loop	24
14	Range-based for Loop temperature conversion	25

15	Modifying an array with a Range-Based for Loop	26
16	Range-based for loop	28
17	Store and display strings using range-based loops	30
18	7.5 Processing array contents	32
19	A payroll program	33
20	Assigning arrays	34
21	Summing the Values in a Numeric Array	36
22	[BONUS] Getting the Average of the Values in a Numeric Array	36
23	[BONUS] Finding the Highest and Lowest Values in a Numeric Array	37
24	7.6 Partially Filled Arrays (Sentinel control)	41
25	[BONUS] Fill array partially from file with sentinel control	42
26	Comparing arrays	44
27	Structured binding declarations	45
28	Using parallel arrays (payroll program revisited)	46
29	Using parallel arrays: Presidents and Vice-Presidents	48
30	Checkpoint (7.6 Parallel arrays)	50
31	TODO 7.7 Arrays as Function Arguments	52
32	Double value of each element of an array	54
33	Processing an array with multiple functions (7-20)	56
34	Checkpoint (7.7 Arrays as function arguments)	65
35	Checkpoint (7.9 Arrays with Three or More Dimensions)	69

36 Case Study (7.10)	75
37 TODO 7.11 Introduction to the STL vector (page 443)	78
38 Declaring vectors	79
39 Storing values in a vector (payroll program revisited)	79
40 Using the range-based for loop with a vector (from C++11)	81
41 Using the <code>push_back</code> member function (payroll pgm modified)	83
42 Determining the size of a vector	85
43 Removing elements from a vector with <code>pop_back</code>	87
44 Clearing a vector	88
45 Detecting an empty vector	89
46 PRACTICE Summary of vector member functions	91
47 Checkpoint (7.11 vectors)	93
48 Glossary	95

1 Objectives

- ☐ Array declaration and initialization
- ☐ Array input without loops
- ☐ Stepping through an array with `for`
- ☐ Running C++ on the command-line with `make`

2 Simple example: Array declaration

- Complete the following program (let's do this together as a warmup):
 1. Supply the definition for an integer array named `values` of size `NUM_ELEMENTS`, and assign the values 100, 200, 300, 400, 500.

2. Print the values of the array on one line separated by a space.

- Starter code: <https://tinyurl.com/array-declaration>

```
// Declare, initialize and print an array of 5 elements
#include <iostream>
using namespace std;

int main()
{
    // Define size of array NUM_ELEMENTS

    // Define int array 'values' of size NUM_ELEMENTS and assign
    // 100...500 to the array

    // Print the array elements on one line separated by a space
    return 0;
}
```

- Solution: <https://onecompiler.com/cpp/43vgaj8dc>

```
#include <iostream>
using namespace std;

int main()
{
    const int NUM_ELEMENTS = 5;
    // Define integer array 'values' of size NUM_DECLARATOR
    // Assign 100...500 to the array
    int values[NUM_ELEMENTS] {100,200,300,400,500};
    // Print the array elements on one line separated by a space
    for (int i=0;i<5;i++) cout << values[i] << " ";

    return 0;
}
```

3 Two types of initialization

- There are two forms of initialization, with or without =.

1. The traditional C-style copy syntax: `value[2] = {100,200};`
 2. The C++ direct list initialization: `value[2] {100,200};`.
- The C++ version is safer because it avoids **narrow conversions**: You cannot initialize an `int` array with `float` values.
 - Demo: This code generates an error

```
float arr[2] = {100,200};
int  arr2[2] = {100., 200.};
```

4 PRACTICE Extended example: Declare and Display Product Prices

You need to upload the URL for this practice file to Canvas.

Scenario

You've joined the IT support team at a small e-commerce startup, Quick-Cart. Your first task is to write a simple C++ program that stores the fixed prices of five featured products. These prices must be stored in an array and displayed on a single line for the company dashboard preview.

Task

- Complete the program below:
 1. Define an floating-point array named `prices` with size `SIZE`.
 2. Assign the values 10.0 100.0 45.99 30.00 200.00 to the array (these represent product prices in dollars).
 3. Print the array elements on one line separated by one space.
- **Expected Output:**

```
#+RESULTS:
: 10 100 45.99 30 200
```

Starter Code: tinyurl.com/array-product-prices

```
// Store product prices in an array
// Author:

#include <iostream>
using namespace std;

int main()
{
    // Set SIZE of array

    // Define array 'prices' and assign product prices

    //Print prices on one line

    return 0;
}
```

Tip

- Remember to use brace list initialization for the array.
- You can use a loop to print each element with a trailing space.

Challenges

1. Let the user enter 5 product prices instead of hardcoding them and prints them out at the end.

Sample output:

```
Enter price for product #1: 10.00
Enter price for product #2: 100.00
Enter price for product #3: 45.99
Enter price for product #4: 30.00
Enter price for product #5: 200.00
Product prices (USD): 10.00 100.00 45.99 30.00 200.00
```

Starter code: tinyurl.com/array-prices-challenge

2. Let user enter 5 product prices instead and validate that each input is a positive number. Otherwise abort with a message.

Sample output:

```
Enter price for product #1: 10
Enter price for product #2: 100
Enter price for product #3: 45.99
Enter price for product #4: 30
Enter price for product #5:
Abort: Negative input.
```

Home assignment!

Solution

- Code: <https://onecompiler.com/cpp/43vgbsg5f>

```
#include <iostream>
using namespace std;

int main()
{
    // set SIZE of array
    const int SIZE = 5;
    // Define array 'prices' and assign product prices
    double prices[SIZE] {10., 100., 45.99, 30., 200.};
    // Print array values on one line separated by one space
    for (int i=0;i<SIZE;i++) cout << prices[i] << " ";
    return 0;
}
```

- Challenge solution 1: <https://onecompiler.com/cpp/43vhsw68n>

Takes input from user, prints all product prices at end. Notice the I/O manipulation library `<iomanip>` and the `stdout` manipulation commands `fixed`, `showpoint` and `setprecision`.

```
// Store product prices in an array: Get five prices as user input
// Sample input: 10 100 45.99 30 -200
// Sample output: Enter price for product #1: 100.00 ...
//                  Enter price for product #5: 200.00
//                  Abort: Negative input.
// Author: Marcus Birkenkrahe (pledged)
```

```

#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    const int SIZE = 5;
    // Define array 'prices'
    double prices[SIZE];
    // Let user enter product prices and print values as entered
    cout << fixed << showpoint << setprecision(2);
    for (int i=0; i<SIZE; i++)
    {
        cout << "Enter price for product #"
              << (i+1) << ": ";
        cin >> prices[i];
        cout << prices[i] << endl;
    }
    // Print array values on one line separated by one space
    cout << "Product prices (USD): ";
    for (int i=0; i<SIZE; i++) cout << prices[i] << " ";
    return 0;
}

```

- Challenge solution 2: <https://onecompiler.com/cpp/43vgcyjh7>

This solution solves both challenge problems and uses a `while` loop.

```

// Store product prices in an array: Get five prices as user input
// Validate that input is positive - otherwise abort with message
// Sample input: 10 100 45.99 30 -200
// Sample output: Enter price for product #1: 100.00 ...
//                  Enter price for product #5:
//                  Abort: Negative input.
// Author: Marcus Birkenkrahe (pledged)

```

```

#include <iostream>
using namespace std;

```

```

int main()
{
    // set SIZE of array
    const int SIZE = 5;
    // define double array
    double prices[SIZE];
    // declare input
    double input;
    // declare loop variable
    int product=0;
    // get user input
    while (cin>>input)
    {
        cout << "Enter price for product #"
              << (product+1) << ": ";
        if (input>=0)
        {
            prices[product] = input;
            cout << prices[product] << "\n";
            product++;
        } else {
            cout << "\nAbort: Negative input.";
            break;
        }
    }
    return 0;
}

```

```

Enter price for product #1: 10
Enter price for product #2: 100
Enter price for product #3: 45.99
Enter price for product #4: 30
Enter price for product #5: 200

```

- Generate InputFile:

```

cd ../data
echo "10. 100. 45.99 30. 200." > inputFile
cat inputFile

```

10. 100. 45.99 30. 200.

5 Inputting array content element-wise

- **Task:** Write a program that asks for the number of hours worked by six employees to be entered on the command line. Store the values in an array, and then print them on one line separated by one space. Constraint: **Do not use loops.**

- **Sample input:** 20 12 40 30 30 15

- **Sample output:**

```
Enter the hours worked by 6 employees:
The hours you entered were: 20 12 40 30 30 15
```

- **C++ starter code:** tinyurl.com/employee-hours

```
// Program asks for number of hours worked by six employees to be
// entered on the command-line. Values are stored in an array and
// printed separated by one space. Do not use loops at all.
// Author:
```

```
// include header file
// use std namespace
// main program
// define array size declarator NUM_EMPLOYEES
// define array 'hours'
// ask user for keyboard input
// get array input from keyboard
// print output
```

- **Solution:** <https://onecompiler.com/cpp/43vgeh56g>

```
// Program asks for number of hours worked by six employees to be
// entered on the command-line. Values are stored in an array and
// printed separated by one space. Do not use loops at all.
// Author: Marcus Birkenkrahe (pledged)
```

```
// include header file
```

```

#include <iostream>
// use std namespace
using namespace std;
// main program
int main()
{
    // define array size declarator
    const int NUM_EMPLOYEES = 6;
    // define array
    int hours[NUM_EMPLOYEES];
    // ask user for keyboard input
    cout << "Enter the hours worked by "
         << NUM_EMPLOYEES << " employees: ";
    // get array input from keyboard
    cin >> hours[0];
    cin >> hours[1];
    cin >> hours[2];
    cin >> hours[3];
    cin >> hours[4];
    cin >> hours[5];
    // print output
    cout << "\nThe hours you entered were:";
    cout << " " << hours[0];
    cout << " " << hours[1];
    cout << " " << hours[2];
    cout << " " << hours[3];
    cout << " " << hours[4];
    cout << " " << hours[5];

    return 0;
}

```

Input file:

```

echo "20 12 40 30 30 15" > input
cat input

```

```

20 12 40 30 30 15

```

6 Stepping through an array

- Change the previous solution to get the keyboard input and generate the output using a loop to step through the array.
- Solution: tinyurl.com/employee-hours-2

```
// Program asks for number of hours worked by six employees to be
// entered on the command-line. Values are stored in an array and
// printed separated by one space. Use for loops.
// Author: Marcus Birkenkrahe (pledged)
// include header file
#include <iostream>
// use std namespace
using namespace std;
// main program
int main()
{
    // define array size declarator
    const int NUM_EMPLOYEES = 6;
    // define array
    int hours[NUM_EMPLOYEES];
    // ask user for keyboard input
    cout << "Enter the hours worked by "
         << NUM_EMPLOYEES << " employees: ";
    // get array input from keyboard with a loop
    for (int i=0; i<NUM_EMPLOYEES; i++)
        cin >> hours[i];
    // print output
    cout << "\nThe hours you entered were:";
    for (int i=0; i<NUM_EMPLOYEES; i++)
        cout << " " << hours[i];
    cout << endl;
    return 0;
}
```

```
echo "20 12 40 30 30 15" > input
```

7 PRACTICE Running C++ on the command-line (CLI)

1. Open the Google Cloud Shell at ide.cloud.google.com
2. Open the terminal
3. Download the source code from the web:

```
wget -O hours.cpp tinyurl.com/employee-hours-cli
```

4. Check the file with `cat hours.cpp`
5. Compile the file with `make hours`
6. Run the file with `./hours` and enter 20 12 40 30 30 15
7. Upload screenshot of your terminal with the result to Canvas.

8 Initializing a string array

- Write a program that **safely** initializes a **string** array named **planets** with the names of the planets of our solar system (ordered by distance from the Sun). Print the result. Make sure that you can tangle the source code file as **planets.cpp** if needed.
- Sample output:

```
#+RESULTS: PLANETS_CPP
: Planets of our solar system:
: Mercury
: Venus
: Earth
: Mars
: Jupiter
: Saturn
: Uranus
: Neptune
```

- Starter code: <https://tinyurl.com/string-array>

```

// Initialize string array 'planets' with the names of the 8 planets
// of our solar system (ordered by distance from the sun). Print it.
// header files
// use std namespace
// main pgm
// define array SIZE
// define and initialize string array planets
// print the names of the planets of our solar system

```

- Solution: <https://onecompiler.com/cpp/43w7wkca9>

```

// Initialize string array 'planets' with the names of the 8 planets
// of our solar system (ordered by distance from the sun). Print it.
// header files
#include <iostream>
#include <string>
// use std namespace
using namespace std;
// main pgm
int main()
{
    // define array SIZE
    const int SIZE=8;
    // define and initialize string array planets
    string planets[SIZE] {"Mercury", "Venus", "Earth",
                          "Mars", "Jupiter", "Saturn",
                          "Uranus", "Neptune"};

    // print the names of the planets of our solar system
    cout << "Planets of our solar system:" << endl;
    for (int count=0; count<SIZE; count++)
        cout << planets[count] << endl;
    return 0;
}

```

```

Planets of our solar system:
Mercury
Venus
Earth

```

Mars
Jupiter
Saturn
Uranus
Neptune

- Note: The `#+NAME: PLANETS_CPP` names the code block. It can now be referenced anywhere under the name `PLANETS_CPP`, and the name also identifies the `#+RESULTS` output.
- With the `:noweb yes` flag, you can reference a named code block also inside a code block:

```
// Initialize string array 'planets' with the names of the 8 planets
// of our solar system (ordered by distance from the sun). Print it.
// header files
#include <iostream>
#include <string>
// use std namespace
using namespace std;
// main pgm
int main()
{
    // define array SIZE
    const int SIZE=8;
    // define and initialize string array planets
    string planets[SIZE] {"Mercury", "Venus", "Earth",
                          "Mars", "Jupiter", "Saturn",
                          "Uranus", "Neptune"};

    // print the names of the planets of our solar system
    cout << "Planets of our solar system:" << endl;
    for (int count=0; count<SIZE; count++)
        cout << planets[count] << endl;
    return 0;
}
```

9 Partial array initialization

- You can partially initialize arrays. Example:

```
const int SIZE = 7;
int num[SIZE]{1,2,3};
for(int i=0;i<SIZE;i++) cout << num[i] << " ";
```

```
1 2 3 0 0 0 0
```

- You can implicitly size arrays if you initialize them with a list. Example:

```
double ratings[]{1.3,6.6,8.4,2.2};
cout << sizeof(ratings)/sizeof(ratings[0]);
```

```
4
```

- An alternative size check uses `std::size` from `<iterator>`:

```
double ratings[]{1.3,6.6,8.4,2.2};
cout << std::size(ratings); // must include <iterator>
```

```
4
```

10 Reading a file into an array

- How can you read the contents of a file into an array?

Open the file and read each item from the file storing each item in an array element until either the array is filled or EOF is reached.

- Example: We put 10 numbers into a file `TenNumbers.txt` and read these numbers into an array.
- Input file:

```
cd ../data
echo "101 102 103 104 105 106 107 108 109 110" > TenNumbers.txt
cat TenNumbers.txt
```

101 102 103 104 105 106 107 108 109 110

```
// file2array.cpp: read data from a file into an array
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    const int ARRAY_SIZE = 10; // Array size
    int numbers[ARRAY_SIZE]{0}; // Array of 10 elements (init 0)
    int count = 0; // Loop counter
    ifstream inputFile; // Input file stream object

    // Open the file
    inputFile.open("../data/TenNumbers.txt");

    // Read numbers from the file into the array
    while (count < ARRAY_SIZE &&
           inputFile >> numbers[count])
        count++; // next array element

    // Close the file
    inputFile.close();

    // Display the numbers read
    cout << "The numbers are: ";
    for (count=0; count<ARRAY_SIZE; count++)
        cout << numbers[count] << " ";

    return 0;
}
```

- The `while` loop test two Boolean conditions, which must both be `true`:
 1. `count < ARRAY_SIZE` so that we don't write past the end of the array.
 2. `inputFile >> numbers[count]` reads value from the file and stores it in an array element. If no value can be read (e.g. because the file is empty or the EOF is reached), the loop stops.

- Test the program with an empty `inputFile`!

```
cd ../data
> TenNumbers.txt
ls -lt TenNumbers.txt
```

```
-rw-rw-r-- 1 aletheia aletheia 0 Oct 30 08:23 TenNumbers.txt
```

11 Writing an array to a file

- How can you write the contents of an array to a file?

Use a loop to step through each element of the array, writing it to file.

- Example:

```
// array2file.cpp: write contents of an array to a file
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    const int ARRAY_SIZE = 10; // array size
    int numbers[ARRAY_SIZE]{0}; // array of 10 elements (init 0)
    int count; // loop counter
    ofstream outputFile; // Output file stream object

    // initialize array
    for (count=0; count < ARRAY_SIZE; count++)
        numbers[count] = count+100;

    // Open a file for output
    outputFile.open("../data/SavedNumbers.txt");

    // Write the array contents to the file
    for (count=0; count < ARRAY_SIZE; count++)
        outputFile << numbers[count] << endl;
```

```

    // Close the file
    outputFile.close();

    // Print success message
    cout << "The numbers were saved to the file.\n";

    return 0;
}

```

- Check file:

```

cd ../data
cat SavedNumbers.txt

```

```

100
101
102
103
104
105
106
107
108
109

```

12 Checkpoint (7.3 No Bounds Checking)

1. Define the following arrays:

- (a) `empNums`, a 100-element array of integers

```
int empNums[100];
```

- (b) `payRates`, a 25-element array of floating-point numbers

```
float payRates[25];
```

- (c) `miles`, a 14-element array of long integers

```
long miles[14];
```

- (d) `cityName`, a 26-element array of string objects

```
std::string cityName[26];
```

- (e) `lightYears`, a 1,000-element array of double-precision floating-point numbers.

```
double lightYears[1000];
```

2. What is wrong with the following array definitions?

```
int readings[-1];  
float measurements[4.5];  
int size; std::string names[size];
```

- (a) `int readings[-1];` - subscript must be positive integer constant.
- (b) `float measurements[4.5];` - subscript must be positive integer constant
- (c) `int size; std::string names[size];` - no variable length arrays in C++ allowed. `size` must be a constant expression at compile-time.

3. What would be the valid subscript values in a 4-element `double` array?

0,1,2,3

4. What is the difference between an array's size declarator and a subscript?

- The **size declarator** specifies how many elements an array can hold — it must be a positive constant integer at compile time.
- A **subscript** (or index) is used to access individual elements of the array. It can vary during program execution, typically in a loop. The valid range of subscripts is from 0 to `SIZE - 1`.

5. What is "array bounds checking"? Does C++ perform it?

- **Array bounds checking** refers to the process of verifying that array subscripts are within the valid range (0 to `size - 1`) during runtime.

- C++ does **not** perform automatic bounds checking for built-in arrays. Accessing out-of-bounds elements results in **undefined behavior**, and it's the programmer's responsibility to ensure safety.

6. What is the output of the following code?

```
int values[5], count;
for (count = 0; count < 5; count++)
    values[count] = count + 1;
for (count = 0; count < 5; count++)
    cout << values[count] << endl;
```

```
1
2
3
4
5
```

7. The following program contains a 10-element array of integers called **fish**. When completed, the program should ask how many fish were caught by fishermen 1 through 10, and store this data in the array. Complete the program, and then test it using a shell code block.

Sample output:

```
#+RESULTS:
:
: Enter catch: 40
: Enter catch: 20
: Enter catch: 33
: Enter catch: 56
: Catch: 40 20 33 56
```

Starter code:

```
#include <iostream>
using namespace std;
int main() {
    const int NUM_FISH = 20;
```

```

    int fish[NUM_FISH];
    // Ask how many fish were caught by fishermen 1-20, and store this
    // data in the array fish, then print the array.

    return 0;
}

```

Testing suite:

```

# change to 'src' directory
# compile fish.cpp as fish
# run fish with the sample input

```

Solution:

```

#include <iostream>
using namespace std;
int main() {
    const int NUM_FISH = 4;
    int fish[NUM_FISH];
    // Enter catch for fishermen 1 through 10
    for (int &howMany : fish) {
        cout << "\nEnter catch: ";
        cin >> howMany;
        cout << howMany;
    }
    cout << "\nCatch: ";
    for (int howMany : fish)
        cout << howMany << " ";
    return 0;
}

```

(org-babel-tangle)

```

cd ../src # change to 'src' directory
g++ fish.cpp -o fish # compile fish.cpp as fish
echo "40 20 33 56" | ./fish # run fish with the sample input

```

```
Enter catch: 40
Enter catch: 20
Enter catch: 33
Enter catch: 56
Catch: 40 20 33 56
```

8. Define the following arrays:

- (a) **ages**, a 10-element array of integers, initialized with the values 5,7,9,14,15,17,18,19,21,23.
- (b) **temps**, a 7-element array of floating-point values, initialized with the values 14.7, 16.3, 18.43, 21.09, 17.9, 18.76, 26.7.
- (c) **alpha**, an 8-element array of characters, initialized with the values J, B, L, A, *, \$, H, M.

Solution:

```
int ages [] {5,7,9,14,15,17,18,19,21,23};
float temps [] {14.7, 16.3, 18.43, 21.09, 17.9, 18.76, 26.7};
char alpha [] {'J', 'B', 'L', 'A', '*', '$', 'H', 'M'};
```

9. Is each of the following a valid or an invalid array definition? If a definition is invalid, explain why.

- (a) Integer array:

```
int numbers[10] = {0, 0, 1, 0, 0, 1, 0, 0, 1, 1}; // valid
```

- (b) Integer array:

```
int matrix[5] = {1,2,3,4,5,6,7}; // invalid - out of bounds
```

- (c) Double precision array:

```
double radii[10] = {3.2, 4.7}; // valid - elements 3 to 10 undefined
```

- (d) Integer array:

```
int table[7] = {2, , , 27, 45, 39}; // invalid - expression missing
```

- (e) Character array:

```
char codes[] = {'A', 'X', '1', '2', 's'}; // valid
```

- (f) Integer array:

```
int blank[]; // invalid - storage size not known
```

13 7.4 The Range-based for loop

- This loop iterates once for each element in an array. It was introduced in C++11.
- General form:

```
for (dataType rangeVariable : array)
    statement
```

- Here, ‘dataType’ is the data type of the ‘rangeVariable’, which receives a different array element during each loop iteration.
- ‘array’ is the array the loop operates on, and ‘statement’ is what you want to do in an iteration.
- Example: Use range-based iteration to display the array: 3,6,9.

```
int numbers[]{3,6,9};
for (int val : numbers)
    cout << val << " ";
```

3 6 9

- You can use the `auto` key word to infer the data type of the range variable:

```
int num[]{3,6,0};
for (auto val : num) cout << val << " ";
```

3 6 0

- Example: Display the elements of a `string` array.

```
string names[]{"Jim","Joe","Jane","Jack"};
for (auto name : names) cout << name << endl;
```

Jim
Joe
Jane
Jack

14 Range-based for Loop temperature conversion

Write a small C++ program that does the following:

1. Define an array of six floating-point temperatures in Celsius: 22.5, 25.0, 19.5, 21.0, 23.5, 26.1.
2. Use a range-based **for** loop with **auto** to:
 - Convert each temperature to Fahrenheit using the formula: $F = C * 9/5 + 32$
 - Print each converted temperature on a new line.
 - Sample output line: "22.5C = 72.5F"

Bonus: Use `std::setprecision` from `<iomanip>` to format the output to one decimal precision.

- Solution:

```
double temps[] = {22.5, 25.0, 19.5, 21.0, 23.5, 26.1};
for (auto temp : temps)
    cout << temp << "C = "
         << temp * 9./5. + 32. << "F." << endl;
```

```
22.5C = 72.5F.
25C = 77F.
19.5C = 67.1F.
21C = 69.8F.
23.5C = 74.3F.
26.1C = 78.98F.
```

- Bonus solution:

1. **fixed:** Forces output to use fixed-point notation (not scientific).
2. **showpoint:** Always show a decimal point, even for whole numbers.
3. **setprecision(1):** Set number of digits after decimal point to one.

```

#include <iomanip>

int main()
{
    double temps[] = {22.5, 25.0, 19.5, 21.0, 23.5, 26.1};
    for (auto temp : temps)
        cout << fixed << showpoint << setprecision(1)
            << temp << "C = "
            << temp * 9./5. + 32. << "F."
            << endl;
    return 0;
}

22.5C = 72.5F.
25.0C = 77.0F.
19.5C = 67.1F.
21.0C = 69.8F.
23.5C = 74.3F.
26.1C = 79.0F.

```

15 Modifying an array with a Range-Based for Loop

- As the range-based loop executes, the range variable receives a copy of an array element. To change the contents of an array, the range variable needs to be declared as a reference.
- What is a reference?

A reference in C++ is an alias for an existing variable: It must be initialized when declared, and it cannot be changed to refer to something else. Any operation on the reference affects the original. Syntax: &

- Code example:

```

int x = 10;
int &ref = x; // x is another name for ref
ref = 20; // changes x
cout << "x = " << x << endl; // prints 20

```

```
x = 20
```

- Example: Declare integer array of `SIZE` 5, get values for the array from `inputFile` in the `../data` directory, and display the array values. Use a range-based for loop for both iterations.

```
const int SIZE{5};
int numbers[5];
ifstream inputFile;

// Get values for the array from input file
inputFile.open("../data/inputFile");
for (int &val : numbers)
    inputFile >> val;

inputFile.close();

// Display values in the array
for (int val : numbers)
    cout << val << " ";
```

```
10 0 0 0 0
```

- Input file:

```
cd ../data
echo "101 102 103 104 105" > inputFile
cat inputFile
```

```
101 102 103 104 105
```

- Why does the first loop use `&val` while the second loop uses `val`?

The first loop modifies the original array - `&val` is an alias for the element that the loop is currently working on.

The second loop only displays the array elements but does not alter them.

- What's the advantage of range-based loops?

When using range-based loops, you do not have to worry about stepping outside of the bounds of an array!

- What's the disadvantage of range-based loops?

When using range-based loops, you do not have access to individual subscripts.

16 Range-based for loop

Starter code: tinyurl.com/range-warmup

- What will be displayed if the user enters the value 5?

```
// Change value of an array using a range-based loop
// Input: 5
// Output: 17
int values[] = {12};
for (int value : values)
{
    cout << "Enter value: ";
    cin >> value;
}
cout << '\n' << values[0] << endl;
```

```
Enter value:
12
```

- Tangle the code: (org-babel-tangle) & build with make

```
cd ../src                                # change to directory ../src
[ -f Makefile ] && rm Makefile             # if exists, remove Makefile
[ -f values.cpp ] && touch values.cpp      # if exists, touch values.cpp
make values                              # build executable 'values'
echo 5 | ./values                         # run 'values' with input 5
```

```
g++      values.cpp  -o values
Enter value:
12
```

- Answer:

The array `values` will not be modified. `value[0]` has the value 12. How would you fix that so that entering 5 prints 5?

```
// Change value of an array using a range-based loop
// Input: 5
// Output: 17
int values[] = {12};
for (int &value : values)
{
    cout << "Enter value: ";
    cin >> value;
}
cout << '\n' << values[0] << endl;
```

```
Enter value:
12
```

- Tangle the code: (org-babel-tangle)

```
cd ../src
[ -f Makefile ] && rm Makefile
[ -f values2.cpp ] && touch values2.cpp
make values2
echo 5 | ./values2
```

```
g++      values2.cpp  -o values2
Enter value:
5
```

17 Store and display strings using range-based loops

- **Task:** Write a C++ program that prompts the user to enter their first, middle, and last names. Store these values in an array of three `string` objects. Then display the full name.
- **Sample input:** Create an `inputFile` using a `bash(1)` shell block and read the string data into the array from that file using the `:cmdline` code block header argument. Sample input:

James Earl Jones

- **Sample output:**

```
#+RESULTS:
: Enter name:
: Enter name:
: Enter name:
: Full name: James Earl Jones
```

- **Starter code:** The program below does this with a regular `for` loop. Modify it to work with a range-based loop instead.
- Link to the starter code: <https://tinyurl.com/range-based>

```
// Get first, middle and last user name and print full name
// Author: Marcus Birkenkrahe (pledged)
#include <iostream>
#include <string>
using namespace std;

int main() {
    const int SIZE {3};      // string array length
    string fullName[SIZE];   // string array declaration

    // get array elements from inputFile
    for (int count=0; count<SIZE; count++) {
        cout << "\nEnter name: ";
        cin >> fullName[count];
    }
}
```

```

    // print result from array
    cout << "\nFull name: ";
    for (int count=0; count<SIZE; count++)
        cout << fullName[count] << " ";

    return 0;
}

```

```

Enter name:
Enter name:
Enter name:
Full name: 101 102 103

```

- Create inputFile:

```

cd ../data
echo "James Earl Jones" > inputFile
cat inputFile

```

```

James Earl Jones

```

- Upload the URL to your solution to Canvas.
- **Solution:** <https://onecompiler.com/cpp/43wcpb8vd>

```

// Get first, middle and last user name and print full name
// Author: Marcus Birkenkrahe (pledged)
#include <iostream>
#include <string>
using namespace std;

int main() {
    const int SIZE {3}; // string array length
    string fullName[SIZE]; // string array declaration

    // get array elements from inputFile
    for (string &name : fullName) {

```

```

        cout << "\nEnter name: ";
        cin >> name;
    }

    // print result from array
    cout << "\nFull name: ";
    for (string name : fullName)
        cout << name << " ";

    return 0;
}

```

```

Enter name:
Enter name:
Enter name:
Full name: James Earl Jones

```

18 7.5 Processing array contents

- Processing array elements is no different than processing other variables. This includes use of pre- and post-increment operations.
- Example: Increase score values in an array

```

int score[] {7,8,9,10,11};
cout << score[2] << endl;
cout << ++score[2] << endl; // pre-increment on value in score[2]=9
cout << score[4]++ << endl; // post-increment on value in score[4]=11
cout << score[4];

```

```

9
10
11
12

```

- An easy mistake is to confuse operations on the value with operations on the subscript:

```

count = 1;
value[count--]; // value[1] -> value[0]
value[count]--; // value[1] -> value[1] - 1

```

19 A payroll program

- Example: This program stores, in an array `hours` the hours worked by `NUM_EMPLOYEES` employees who all make the same hourly wage `payrate`. The program computes the `grossPay` for each employee as the product of `payrate` and the `hours` worked by the employee.

```

#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    const int NUM_EMPLOYEES = 5; // Number of employees
    int hours[NUM_EMPLOYEES];    // Array to hold hours
    double payrate;              // Hourly pay rate
    double grossPay;             // To hold gross pay

    // Input hours worked
    cout << "Enter hours worked by " << NUM_EMPLOYEES
         << " employees who all earn the same hourly rate.\n" << endl;
    for (int index = 0; index < NUM_EMPLOYEES; index++) {
        cout << "Employee #" << (index + 1) << ": ";
        cin >> hours[index];
        cout << hours[index] << endl;
    }

    // Input the hourly rate for all employees
    cout << "\nEnter the hourly pay rate for all the employees: ";
    cin >> payrate;
    cout << payrate;

    // Display each employee's gross pay
    cout << "\nHere is the gross pay for each employee:\n";
    cout << fixed << showpoint << setprecision(2);
    for (int index = 0; index < NUM_EMPLOYEES; index++) {

```

```

        grossPay = hours[index] * payrate;
        cout << "Employee #" << (index + 1);
        cout << ": $" << grossPay << endl;
    }
    return 0;
}

```

- Tangle the code (org-babel-tangle) and test it with a sample input:

```

cd ../src
g++ pay.cpp -o pay
echo 5 10 15 20 40 12.75 | ./pay

```

Enter hours worked by 5 employees who all earn the same hourly rate.

```

Employee #1: 5
Employee #2: 10
Employee #3: 15
Employee #4: 20
Employee #5: 40

```

```

Enter the hourly pay rate for all the employees: 12.75
Here is the gross pay for each employee:
Employee #1: $63.75
Employee #2: $127.50
Employee #3: $191.25
Employee #4: $255.00
Employee #5: $510.00

```

20 Assigning arrays

- Does the following code work? Why or why not?

```

const int SIZE {4};
int oldValues[SIZE] {10,100,200,300};
int newValues[SIZE];
newValues = oldValues;

```

- Answer:

The only way to assign one array to another is to assign the individual elements in the array:

```
const int SIZE {4};
int oldValues[SIZE] {10,100,200,300};
int newValues[SIZE];

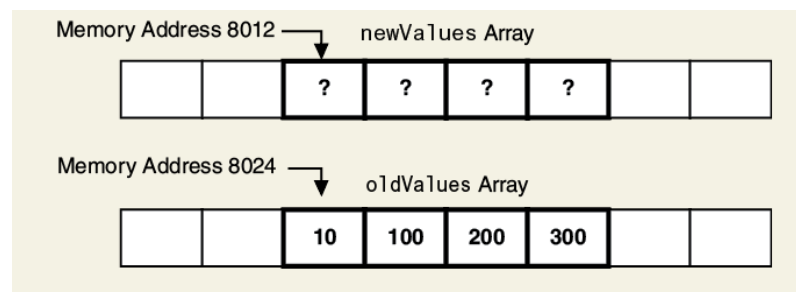
for (int count=0;count<SIZE;count++) {
    newValues[count]=oldValues[count];
    cout << newValues[count] << " ";
}
```

10 100 200 300

- Explanation:

The reason why arrays cannot be assigned to one another is that you'd have to change the starting memory address of the array - because whenever the name of an array is used without subscript, it's the same as the array's starting address, `&newValues[0]`.

In the sample layout, the statement `newValues = oldValues` is seen as `8024 = 8012`.



- If `num` is an array of `int` numbers, what will `cout << num;` print?

```
int num[] {10,20,30,40,50};
cout << num;
cout << '\n' << &num[1]; // memory address of num[1]
```

```
0x7ffc7f5ca360
0x7ffc7f5ca364
```

- Notice that the address of `num[1]` is exactly 4 bytes forward in memory from the address of `num[0]`.

21 Summing the Values in a Numeric Array

- **Task:** Write a short C++ program that sums the values in an array. Use a range-based loop (5 lines.)
- **Sample input:** 1,2,3,4,5,6,7,8
- **Sample output:**

```
#+RESULTS:
: Sum = 36
```

- **Solution:**

```
int sum = 0;
int arr[] {1,2,3,4,5,6,7,8};
for (int val : arr)
    sum += val;
cout << "Sum = " << sum;
```

```
Sum = 36
```

22 [BONUS] Getting the Average of the Values in a Numeric Array

- **Task:** Write a short C++ program that sums the values in the array `scores`. The first step towards the average is to sum up all the values in the array, and the second is to divide by the length of the array.
- **Sample input:** 10,22,34,42,50,66,71,110
- **Sample output:**

```
Average = 36/6 = 6
```

- Submit the OneCompiler.com URL to your solution showing the exact sample output, or the equivalent Emacs Org-mode file.
- **Solution:**

```
const int SIZE = 8;
int scores[SIZE]={10,22,34,42,50,66,71,110};

int sum = 0;
double average;

for (int score : scores)
    sum+=score;

average = static_cast<double>(sum)/SIZE;

cout << "Average = " << sum << "/" << SIZE << " = " << average;

Average = 405/8 = 50.625
```

23 [BONUS] Finding the Highest and Lowest Values in a Numeric Array

- **Task:** Write a short C++ program that finds the highest and the lowest value in a numeric array, **values**, with the values 1,2,3,4,5,6,7,8.
- **Idea:** To find the highest value, assume that the first value is already highest, then step through the array comparing each element and replacing it with the highest if it is found to be higher.
- **Sample output:**

```
Max value = 8
Min value = 1
```

- **Submit** the OneCompiler.com URL to your solution showing the exact sample output, or the equivalent Emacs Org-mode file.
- **Note:** You might want to look at edge cases when you've got your program working - What if all values are identical, if the array is empty, if some values are negative, or if a value is too large (or small) to be displayed?

- **Solution:**

```
#include <iostream>
using namespace std;

int main()
{
    const int SIZE {8};
    int values[SIZE]={1,2,3,4,5,6,7,8};

    int max = values[0];
    int min = values[0];

    for (int i=1;i<SIZE;i++)
    {
        if (values[i] > max) max = values[i];
        if (values[i] < max) min = values[i];
    }
    cout << "Max value = " << max << endl
         << "Min value = " << min << endl;

    return 0;
}
```

```
Max value = 8
Min value = 1
```

- Only the max value:

```
const int SIZE {8};
int values[SIZE]={1,2,3,4,5,6,7,8};

int max = values[0];

for (int i=1;i<SIZE;i++)
    if (values[i] > max) max = values[i];

cout << "Max value = " << max;
```

Max value = 8

- It is worth looking at edge cases:
 1. What if all values of the array are identical?
 2. What if the array is empty?
 3. What if some values are negative?
 4. What if one of values cannot be represented?
- Modified version for edge cases: Get the array values from the command-line (tangle the file `max.cpp`, compile and run it).

```
const int SIZE {8};
int values[SIZE] {0};

// get array values
for (int &val : values) {
    cin >> val;
    cout << val << " ";
}
int max = values[0];

for (int i=1;i<SIZE;i++)
    if (values[i] > max) max = values[i];

cout << "\nMax value = " << max;
```

- Test it:

```
cd ../src
g++ max.cpp -o max
# echo "1 1 1 1 1 1 1 1" | ./max # this edge case works
echo "1 1 1 -1 100 1 1 1" | ./max # this edge case works
# echo "" | ./max # this edge case works
# echo "1 1 1 1 10000000000 1 1 1" | ./max # this edge case does not work
```

```
1 1 1 -1 100 1 1 1
Max value = 100
```

- What would you have to do to make the last edge case work?

```
const int SIZE {8};
int values[SIZE] {0};

// get array values
for (int &val : values) {
    cin >> val;
    cout << val << " ";
}
long max = values[0];

for (int i=1;i<SIZE;i++)
    if (values[i] > max) max = values[i];

cout << "\nMax value = " << max;
```

- Test it:

```
cd ../src
g++ max.cpp -o max
# echo "1 1 1 1 1 1 1 1" | ./max # this edge case works
# echo "1 1 1 -1 100 1 1 1" | ./max # this edge case works
# echo "" | ./max # this edge case works
echo "1 1 1 1 10000000 1 1 1" | ./max # this edge case does not work
```

```
1 1 1 1 10000000 1 1 1
Max value = 10000000
```

- What is the largest value that can be represented as an `int`?

```
#include <climits>
cout << INT_MAX << endl; // see cppreference - search e.g. for "Maximum integer"

2147483647
```

- What would you have to change to get the **lowest** value?

Apart from renaming the variable from `max` to `min`, only one operator has to be changed: `if (values[i] < min) min = values[i];`

```
const int SIZE {8};
int values[SIZE]={1,2,3,4,5,6,7,8};

int min = values[0];

for (int i=1;i<SIZE;i++)
    if (values[i] < min) min = values[i];

cout << "Min value = " << min;

Min value = 1
```

24 7.6 Partially Filled Arrays (Sentinel control)

- What if you want to store items in an array but you don't know how many items there are exactly?

You could make the array large enough to hold the largest possible number of items. As a result, the array will only be partially filled - and only elements with valid data items must be processed.

- Example: The `count` variable acts like a sentinel. We'll tangle and test this below.

```
const int SIZE {100};
int numbers[SIZE];
int count = 0;
int num;

// fetch first number
cout << "Enter a number or -1 to quit: ";
cin >> num;
// fetch next numbers (until full or quit)
while (num != -1 && count < SIZE) {
```

```

        count++; // increase sentinel value
        numbers[count-1] = num;
        cout << "\nEnter a number or -1 to quit: ";
        cin >> num;
    }
    // display valid array items
    for (int index=0; index < count; index++)
        cout << numbers[index] << " ";

```

- Test it:

```

cd ../src
g++ part.cpp -o part
echo "1 3 4 77 8 33 -1" |./part

```

```

Enter a number or -1 to quit:
Enter a number or -1 to quit:
Enter a number or -1 to quit:
Enter a number or -1 to quit:
Enter a number or -1 to quit:
Enter a number or -1 to quit:
Enter a number or -1 to quit: 1 3 4 77 8 33

```

25 [BONUS] Fill array partially from file with sentinel control

- **Task:** Write a C++ program `sentinel.cpp` that reads an unknown number of items from a file `numbers.txt` into an array `numbers` of `SIZE 100`.
- **Sample input:** 47 89 65 36 12 25 17 8 62 10 87 62

```

cd ../data
echo "47 89 65 36 12 25 17 8 62 10 87 62" > numbers.txt
cat numbers.txt

```

```

47 89 65 36 12 25 17 8 62 10 87 62

```

- **Sample output:**

```
47 89 65 36 12 25 17 8 62 10 87 62
```

- **Tip:** To read data from a file:

```
#include <fstream>
ifstream inputFile;
inputFile.open("numbers.txt");
// ...
inputFile.close();
```

- **Starter code:** Pseudocode <https://tinyurl.com/fill-array-file>

```
// Read an unknown number of items from a file 'numbers.txt' into
// an array 'numbers' fixed at SIZE 100.
// header files
// namespace use
// main pgm
// declarations (array size, array, sentinel, input file)
// open input file
// read input into array (while count < SIZE and EOF not reached)
// close input file
// display numbers read
```

- You won't be able to test this on OneCompiler.com but only in Google Cloud Shell (ide.cloud.google.com). Use the nano editor to create the file `numbers.txt` and the source file `sentinel.cpp`.
- Submit a screenshot of your solution with the exact sample output, or the equivalent Emacs Org-mode file. To view the file in the terminal, and to compile/run the executable, use the commands:

```
cat sentinel.cpp # view file
g++ sentinel.cpp -o sentinel # compile and produce executable
./sentinel # run executable
```

- **Solution:** open <https://onecompiler.com/cpp/43wdwc8pt>

```

// header files
#include <iostream>
#include <fstream>
// namespace use
using namespace std;
// main pgm
int main() {
    // declarations (array size, array, sentinel, input file)
    const int SIZE {100};
    int numbers[SIZE];
    int count=0; // sentinel
    ifstream inputFile; // input file
    // open inputFile
    inputFile.open("../data/numbers.txt");
    // read input into array (while count < SIZE and EOF not reached)
    while (count < SIZE && inputFile >> numbers[count]) {
        count++;
        cin >> numbers[count];
    }
    // close input file
    inputFile.close();
    // display numbers read
    for (int index=0; index < count; index++)
        cout << numbers[index] << " ";
    return 0;
}

```

47 89 65 36 12 25 17 8 62 10 87 62

26 Comparing arrays

- Just like you cannot assign an array to another one you can also not compare a whole array to another one.
- What does the following code check and what is the output:

```

int firstArray [] {5,10,15,20,25};
int secondArray [] {5,10,15,20,25};
(firstArray == secondArray) ? (cout<<"same"):(cout<<"different");

```

different

- Answer:

```
int firstArray [] {5,10,15,20,25};
int secondArray [] {5,10,15,20,25};
cout << firstArray << endl << secondArray;
```

```
0x7ffc2ad921c0
0x7ffc2ad921e0
```

- To compare the contents of two arrays, you must compare their elements.

```
const int SIZE {5};
int firstArray [] {5,10,15,20,25};
int secondArray [] {5,10,15,20,25};
bool arraysEqual = true; // flag variable
int count = 0; // loop counter variable

// Determine whether elements contain same data
while (arraysEqual && count < SIZE) {
    if (firstArray[count] != secondArray[count])
        arraysEqual = false;
    count++;
}
// Print result
cout << boolalpha << "Arrays equal? " << arraysEqual;
```

Arrays equal? true

27 Structured binding declarations

- This is a new feature with C++17: A structured binding declaration lets you write one statement that defines a set of variables and initializes those variables with the values that are stored in an array. This is called "unpacking" the array:

- General form: The **array** must have N elements if we want to unpack N variables.

```
auto [variable1, variable2, ..., variableN] = array;
```

- Example:

```
int testScores[] {80, 90, 100};
auto [score1, score2, score3] = testScores;
double average = (score1 + score2 + score3) / 3.0;
cout << "Test score average = " << average;
```

```
Test score average = 90
```

28 Using parallel arrays (payroll program revisited)

- Using the same subscript, you can build relationships between data stored in two or more arrays.
- Example: A variation of the payroll ??: we use two parallel arrays, one for **hours** worked by each employee, another one for the **payRate** paid to each employee. The subscript maintains the identity of the employee.
- For later reference: We could pick different approaches:

Approach	Description	Pros
Add a third array empID	Track employee ID explicitly	Simple; easy upgrade
Use a 2D matrix	Rows as employees, columns pay, hours	Compact; indexable
Use a struct	Group id , hours , rate into one unit	Clear, extensible, readable
Use a class	Add methods, encapsulate behavior	Best for large systems

- Using a **struct** is the best way:
 1. Data are bundled with minimal syntax.
 2. No behavior assumed (no methods needed like **computePay**).
 3. All members are **public** by default.
- Code example:

```

// Keep track of hours worked and pay rates paid to employees
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    const int NUM_EMPLOYEES {5};
    int hours[NUM_EMPLOYEES];
    double payRate [NUM_EMPLOYEES];
    double grossPay;

    // Input hours worked and hourly pay rate
    cout << "Enter the hours worked by " << NUM_EMPLOYEES
        << " employees and their\n"
        << "hourly pay rates.\n";
    for(int index = 0 ; index < NUM_EMPLOYEES ; index++) {
        cout << "\nHours worked by employee #" << (index + 1) << ": ";
        cin >> hours[index];
        cout << hours[index];
        cout << "\nHourly pay rate for employee #" << (index + 1) << ": ";
        cin >> payRate[index];
        cout << payRate[index];
    }

    // Display each employee's gross pay
    cout << "\nHere is the gross pay for each employee:\n";
    cout << fixed << showpoint << setprecision(2);
    for (int index = 0; index < NUM_EMPLOYEES; index++) {
        grossPay = hours[index] * payRate[index];
        cout << "Employee #" << (index + 1);
        cout << ": $" << grossPay << endl;
    }
    return 0;
}

```

- Test it:

```

cd ../src
g++ payroll.cpp -o pay
echo "10 9.75 15 8.62 20 10.50 40 18.75 40 15.65" | ./pay

```

```

Enter the hours worked by 5 employees and their hourly rates.
Hours worked by employee #1: 10Hourly pay rate for employee #1: 9.75Hours worked by
Here is the gross pay for each employee:
Employee #1: $97.50
Employee #2: $129.30
Employee #3: $210.00
Employee #4: $750.00
Employee #5: $626.00

```

- Visualization of the concepts of parallel arrays:

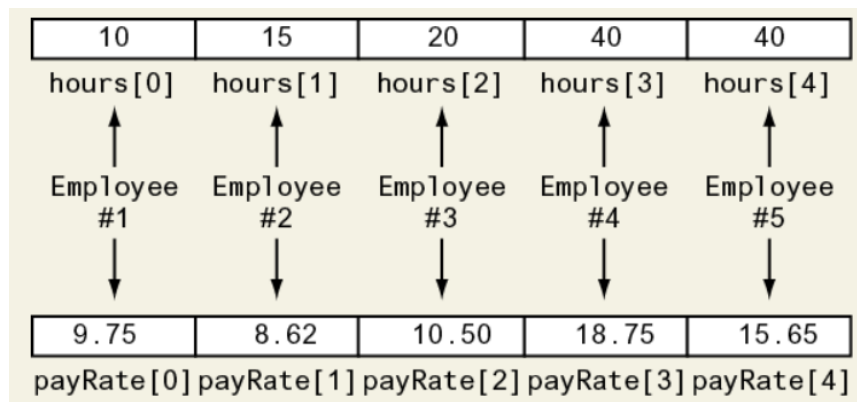


Figure 1: Parallel arrays (Source: Gaddis 2022, p. 419)

29 Using parallel arrays: Presidents and Vice-Presidents

- The `presidents` array contains the names of the first four U.S. presidents, and the `vicePresident` array contains the names of the first four U.S. vice presidents.
- **Task:** Complete the starter code below, compile and run.
- **Sample output:**

```

#+RESULTS:
: George Washington, John Adams
: John Adams, Thomas Jefferson
: Thomas Jefferson, Aaron Burr
: James Madison, George Clinton

```

- Starter code:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    const int SIZE = 4;

    // Define the presidents array here.
    string presidents[SIZE] = {
        "George Washington",
        "John Adams",
        "Thomas Jefferson",
        "James Madison"
    };

    // Define the vicePresidents array here.
    string vicePresidents[SIZE] = {
        "John Adams",
        "Thomas Jefferson",
        "Aaron Burr",
        "George Clinton"
    };

    // Display the presidents and their vice presidents.

    return 0;
}
```

- Solution:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    const int SIZE = 4;

    // Define the presidents array here.
```

```

string presidents[SIZE] = {
    "George Washington",
    "John Adams",
    "Thomas Jefferson",
    "James Madison"
};

// Define the vicePresidents array here.
string vicePresidents[SIZE] = {
    "John Adams",
    "Thomas Jefferson",
    "Aaron Burr",
    "George Clinton"
};

// Complete this loop so it displays the presidents and their vice
// presidents.
for (int count = 0; count < SIZE; count++) {
    cout << presidents[count] << ", "
        << vicePresidents[count] << endl;
}
return 0;
}

```

```

George Washington, John Adams
John Adams, Thomas Jefferson
Thomas Jefferson, Aaron Burr
James Madison, George Clinton

```

30 Checkpoint (7.6 Parallel arrays)

1. Given the array definition `int values [] = {2, 6, 10, 14};` what is the output of the `cout` statements below?

```

int values [] = {2, 6, 10, 14};

cout << values[2] << endl;
cout << ++values[0] << endl;
cout << values[1]++ << endl;

```

```
int x = 2;
cout << values[++x] << endl;
cout << values[x++] << endl;
```

```
10
3
6
14
14
```

2. Given `int nums[5]={1,2,3};`, what will `cout << nums[3]` display?

```
int nums[5]={1,2,3};
cout << nums[3];
```

```
0
```

Explanation:

Uninitialized elements in a global or static array are zero-initialized.

3. What is the output of the following code (do it in your head first)?

```
double balance[5] = {100.0, 250.0, 325.0, 500.0, 1100.0};
const double INTRATE = 0.1;
```

```
cout << fixed << showpoint << setprecision(2);
for (double val : balance)
    cout << val * INTRATE << " ";
```

```
10.00 25.00 32.50 50.00 110.00
```

4. What is the output of the following code (do it in your head first)?

```

const int SIZE = 5;
int time[SIZE] = { 1, 2, 3, 4, 5},
    speed[SIZE] = {18, 4, 27, 52, 100},
    dist[SIZE];

for (int count = 0; count < SIZE; count++) {
    dist[count] = time[count] * speed[count];
    cout << time[count] << " "
         << speed[count] << " "
         << dist[count] << endl;
}

1 18 18
2 4 8
3 27 81
4 52 208
5 100 500

```

31 TODO 7.7 Arrays as Function Arguments

- To pass an array as an argument to a function, pass the name of the array. The name will decay to a pointer to the first array element.
- Example: showValues of an array

```

#include <iostream>
using namespace std;

void showValues(const int [], int); // prototype

int main()
{
    const int ARRAY_SIZE=8;
    int numbers[ARRAY_SIZE]={5,10,15,20,25,30,35,40};
    showValues(numbers,ARRAY_SIZE);
    return 0;
}

// function definition
void showValues(const int num[], int size)

```

```

{
    for (int i=0; i<size; i++)
        cout << num[i] << " ";
}

```

- Wouldn't a range-based loop work in `showValues`, too?

```

void showValues2(int num[])
{
    for (int val : num)
        cout << val;
}

```

- Answer:

The range-based loop inside `showValues` is invalid because it has no end for the range: `ARRAY_SIZE` is not passed, and even if it were passed, it could not be used in the loop. In C++, use `std::array` or `std::vector` instead, which stores size information.

- Note: If an array has been passed as an argument into the function, you cannot use structured binding declaration in the receiving function to unpack the array (since only the beginning of the array is available to the compiler).
- Don't believe me? Let's try it:

```

#include <iostream>
using namespace std;

void showValues2(const int []); // prototype

int main()
{
    const int ARRAY_SIZE=8;
    int numbers[ARRAY_SIZE] {5,10,15,20,25,30,35,40};
    showValues2(numbers); // function call
    return 0;
}
// function definition

```

```

void showValues2(int num[])
{
    for (int val : num)
        cout << val;
}

```

- Result (some stuff deleted):

```

In function 'void showValues2(int*)':
23:18: error: 'begin' was not declared in this scope;
did you mean 'std::begin'?
   23 |     for (int val : num)
       |                ^~~
       |                std::begin
/usr/include/c++/11/bits/range_access.h:108:37:
note: 'std::begin' declared here
   template<typename _Tp> const _Tp* begin(const valarray<_Tp>&) noexcept;
       |                ^~~~~~
23:18: error: 'end' was not declared in this scope; did you mean 'std::end'?
   23 |     for (int val : num)
       |                ^~~
       |                std::end
/usr/include/c++/11/bits/range_access.h:110:37:
note: 'std::end' declared here
   template<typename _Tp> const _Tp* end(const valarray<_Tp>&) noexcept;
       |                ^~~

```

32 Double value of each element of an array

- Write a C++ program that double the value of each element of an array `set` of 7 elements.
- Use functions `doubleArray` to double the value of `set`, and `showValues` to show the elements of the array before and after the doubling.
- Solution:

```

#include <iostream>
using namespace std;

```

```

void doubleArray(int [], int);
void showValues(const int [], int);

int main()
{
    const int SIZE = 7;
    int set[SIZE] = {1,2,3,4,5,6,7};
    showValues(set,SIZE);
    doubleArray(set,SIZE);
    showValues(set,SIZE);
    return 0;
}

void doubleArray(int arr[], int size)
{
    for(int i=0;i<size;i++)
        arr[i]*=2;
}

void showValues(const int arr[], int size)
{
    for(int i=0;i<size;i++)
        cout << arr[i] << " ";
    cout << endl;
}

```

```

1 2 3 4 5 6 7
2 4 6 8 10 12 14

```

- From this result, when an array is passed as an argument to a function, is it **passed by value** or **passed by reference**?

An array is passed by reference: The address of the first array element in memory (a pointer value) is passed to the function.

- Why don't we declare the 2nd argument, `size_of_array`, also `const`?

```

void printArray(const int array[], int size_of_array) {}

```

Regular variables like `size_of_array` are passed by value, which is why `size_of_array` is only a local copy of the argument in the calling routine anyway.

33 Processing an array with multiple functions (7-20)

- **Task:** Dr. LaClaire gives four exams during the semester in her chemistry class. At the end of the semester, she drops each student's lowest test score before averaging the scores. She has asked you to write a program that will read a student's four test scores as input, and calculate the average with the lowest score dropped.

- **Pseudocode:**

```
// Read the student's four test scores.  
  
// Calculate the total of the scores.  
  
// Find the lowest score.  
  
// Subtract the lowest score from the total.  
  
// Divide the adjusted total by 3 (adjusted number of exams).  
  
// Display the average score.
```

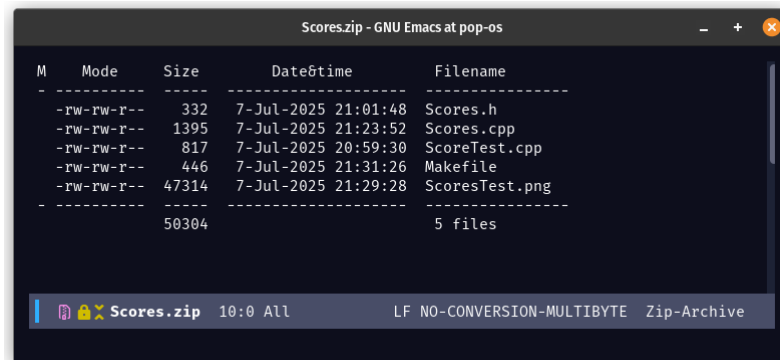
- **Sample input:** Test scores 92, 67, 75, 88.

- **Sample output:**

```
#+RESULTS: 7-20 development version  
:  
: Enter exam score number 1: 92  
: Enter exam score number 2: 67  
: Enter exam score number 3: 75  
: Enter exam score number 4: 88  
: The average with the lowest score dropped is 85.0.
```

- **Submit:** A working program `AverageScores.cpp` in one code block with the sample output.

- **Bonus submit:** An archive `Scores.zip` with a header file `Scores.h` (containing `#include` commands, `namespace` command, and the function prototype declarations), the function definitions file `Scores.cpp`, the test file with the `main` function, `ScoresTest.cpp`, a `Makefile`, and a screenshot of a successful run of the program on the command-line, `ScoresTest.png`.



- **Edge cases:**

1. What if all exam scores are identical (but non-zero)?
2. What if one exam score is zero (so it could be dropped without doing any computations)?

Answer: If all scores are the same (but non-zero), the algorithm still works - one of the scores is dropped. If one score is zero, then it will be dropped. One could make the call of `minScore` conditional but all other computations still have to be done.

- **Solution:**

A useful first step: Turn all actions into modular functions. Prototype and define them (without doing anything yet). Check syntax.

```
#include <iostream>
using namespace std;
```

```
// Get student's exam scores and store them in an array.
void getScore(int [], int);
```

```

// Calculate the total by adding array elements.
int sumScore(int [], int);
// Find the lowest score.
int minScore(int[], int);
// Adjust the score by subtracting the lowest score.
void adjustScore(int [], int);
// Average the adjusted score.
int averageScore(int [], int);
// Display the average score.
void printScore(int [], int);

int main()
{
    const int SIZE = 4;
    int score[SIZE];

    // Read the student's four test scores.
    getScore(score, SIZE);

    // Calculate the total of the scores.
    sumScore(score, SIZE);

    // Find the lowest score.
    minScore(score, SIZE);

    // Subtract the lowest score from the total.
    adjustScore(score, SIZE);

    // Divide the adjusted total by 3 (adjusted number of exams).
    averageScore(score, SIZE);

    // Display the average score.
    printScore(score, SIZE);

    return 0;
}

// Get student's exam scores and store them in an array.
void getScore(int num[], int size)
{

```

```

}
// Calculate the total by adding array elements.
int sumScore(int num[], int size)
{
    int sum=0;
    return sum;
}
// Find the lowest score.
int minScore(int[], int)
{
    int min=0;
    return min;
}
// Adjust the score by subtracting the lowest score.
void adjustScore(int num[], int size)
{
}
// Average the adjusted score.
int averageScore(int num[], int size)
{
    int mean=0;
    return mean;
}
// Display the average score.
void printScore(int num[], int size)
{
}

```

Complete and test function-by-function (with input file and `:cmdline`). An alternative is to tangle the file, compile it with the `-g` option and run it through `gdb`.

Insight: Don't need the functions `adjustScore`, `averageScore`, and `printScore`.

```

echo "92 67 75 88" > input
cat input

```

```

92 67 75 88

```

```

#include <iostream>
#include <iomanip>
using namespace std;

// Get student's exam scores and store them in an array.
void getScore(int [], int);
// Calculate the total by adding array elements.
int sumScore(int [], int);
// Find the lowest score.
int minScore(int[], int);

int main()
{
    const int SIZE = 4; // number of exams
    int score[SIZE];    // holds exam scores
    int total;          // sum of exam scores
    int min;            // lowest exam score
    float average;      // average of adjusted total

    // Read the student's four test scores.
    getScore(score, SIZE);

    // Calculate the total of the scores.
    total = sumScore(score, SIZE);

    // Find the lowest score.
    min = minScore(score, SIZE);

    // Subtract the lowest score from the total.
    total -= min;

    // Divide the adjusted total by 3 (adjusted number of exams).
    average = total/(SIZE-1);

    // Display the average score.
    cout << fixed << showpoint << setprecision(1);
    cout << "\nThe average with the lowest score dropped is "
         << average << "." << endl;

    return 0;
}

```

```

}

// Get student's exam scores and store them in an array.
void getScore(int num[], int size)
{
    for (int i=0; i<size; i++)
    {
        cout << "\nEnter exam score number "
              << (i + 1) << ": ";
        cin >> num[i];
        cout << num[i];
    }
}

// Calculate the total by adding array elements.
int sumScore(int num[], int size)
{
    int sum=0;
    for (int i=0; i<size; i++)
    {
        sum += num[i];
    }
    return sum;
}

// Find the lowest score.
int minScore(int num[], int size)
{
    int min=num[0];
    for (int i=1; i<size; i++)
    {
        if (num[i] < min) min = num[i];
    }
    return min;
}

```

```

Enter exam score number 1: 92
Enter exam score number 2: 67
Enter exam score number 3: 75
Enter exam score number 4: 88

```

The average with the lowest score dropped is 85.0.

- **Bonus solution:** For the bonus solution, we split the code up in several files, and document well (more important than when everyone is in one file).

Remember: You only want to include the header file once (wrap in `#ifndef...#endif` macro statements). The function definition file and the test file (with `main`) only include the header file.

1. Header file: `Scores.h`

```
#ifndef SCORES_H
#define SCORES_H
#include <iostream>
#include <iomanip>
using namespace std;

// Get student's exam scores and store them in an array.
void getScore(double [], int);
// Calculate the total by adding array elements.
double sumScore(double [], int);
// Find the lowest score.
double minScore(double [], int);

#endif
```

2. Function file: `Scores.cpp`

```
#include "Scores.h"

//*****
// The getScore function accepts a double array and its *
// size as arguments. It prompts the user to enter test *
// scores, which are stored in the array. *
//*****
void getScore(double num[], int size)
{
    for (int i=0; i<size; i++)
    {
        cout << "Enter exam score number "
```

```

        << (i + 1) << ": ";
        cin >> num[i];
    }
}

//*****
// The sumScore function accepts a double array and its *
// size as arguments. The sum of the array's elements is *
// returned as a double. *
//*****
double sumScore(double num[], int size)
{
    double sum=0.;
    for (int i=0; i<size; i++)
    {
        sum += num[i];
    }
    return sum;
}

//*****
// The minScore function accepts a double array and its *
// size as arguments. The lowest value in the array is *
// returned as a double. *
//*****
double minScore(double num[], int size)
{
    double min=num[0];
    for (int i=1; i<size; i++)
    {
        if (num[i] < min) min = num[i];
    }
    return min;
}

```

3. Test file: ScoreTest.cpp

```

#include "Scores.h"

int main()
{

```

```

const int SIZE = 4; // number of exams
double score[SIZE]; // holds exam scores
double total;       // sum of exam scores
double min;         // lowest exam score
double average;     // average of adjusted total

// Read the student's four test scores.
getScore(score, SIZE);

// Calculate the total of the scores.
total = sumScore(score, SIZE);

// Find the lowest score.
min = minScore(score, SIZE);

// Subtract the lowest score from the total.
total -= min;

// Divide the adjusted total by 3 (adjusted number of exams).
average = total/(SIZE-1);

// Display the average score.
cout << fixed << showpoint << setprecision(2);
cout << "\nThe average with the lowest score dropped is "
    << average << "." << endl;

return 0;
}

```

4. Makefile: Insert the TAB in a code block with C-q TAB.

```

# Makefile for ScoreTest.cpp
CXX      = g++
CXXFLAGS = -std=c++17 -Wall -Wextra -g

TARGET   = ScoreTest
OBJS     = Scores.o ScoreTest.o

$(TARGET): $(OBJS)
$(CXX) $(CXXFLAGS) -o $(TARGET) $(OBJS)

```

```

Scores.o: Scores.cpp Scores.h
$(CXX) $(CXXFLAGS) -c Scores.cpp

ScoreTest.o: ScoreTest.cpp Scores.h
$(CXX) $(CXXFLAGS) -c ScoreTest.cpp

clean:
rm -f *.o $(TARGET)

zip:
zip Scores.zip Scores.h Scores.cpp ScoreTest.cpp Makefile ScoresTest.png

```

34 Checkpoint (7.7 Arrays as function arguments)

1. Given the following array definitions, will the statement work? If not, why not?

```

double array1[4] = {1.2, 3.2, 4.2, 5.2};
double array2[4];
array2 = array1;

```

The assignment will not work because you cannot assign one array to another. You have to copy the array over element-wise with a loop.

2. When an array is passed to a function, what is actually being passed?

When an array is passed to a function, the address of the first array element is passed (in the form of a pointer to that value).

3. When used as function arguments, are arrays passed by value?

When used as function arguments, arrays are passed by reference (to their first element).

4. What is the output of the following program?

```

#include <iostream>
using namespace std;

```

```

void fillArray(char [], int);
void showArray(const char [], int);

int main()
{
    const int SIZE = 8;
    char prodCode[SIZE] {'0','0','0','0','0','0','0','0'};

    fillArray(prodCode, SIZE);
    showArray(prodCode, SIZE);
    return 0;
}

// Definition of function fillArray
// (Hint: 65 is the ASCII code for 'A')
void fillArray(char arr[], int size)
{
    char code = 65;
    for (int k = 0; k < size; code++, k++)
        arr[k] = code;
}

// Definition of function showArray
void showArray(const char codes[], int size)
{
    for (int k = 0; k < size; k++)
        cout << codes[k];
    cout << endl;
}

```

ABCDEFGH

5. The following program skeleton, when completed, will ask the user to enter 10 integers, which are stored in an array. The function **avgArray**, which you must write, is to calculate and return the average of the numbers entered.

Tangle and test the program **avgArray.cpp** using a shell code block (compilation and input of the numbers 1,2,3,4,5,6,7,8,9,10).

```

#include <iostream>
using namespace std;

// Write your function prototype here

int main()
{
    const int SIZE = 10;
    int userNums[SIZE];
    cout << "Enter 10 numbers: ";
    for (int count = 0; count < SIZE; count++)
    {
        cout << "#" << (count + 1) << " ";
        cin >> userNums[count];
    }
    cout << "The average of those numbers is "
         << avgArray(userNums, SIZE) << endl;
    return 0;
}

// Write your function definition here

```

Solution:

```

#include <iostream>
using namespace std;

// Write your function prototype here
double avgArray(int [], int);

int main()
{
    const int SIZE = 10;
    int userNums[SIZE];
    cout << "Enter 10 numbers:\n";
    for (int count = 0; count < SIZE; count++)
    {
        cout << "#" << (count + 1) << " ";
        cin >> userNums[count];
        cout << userNums[count] << endl;
    }
}

```

```

    }
    cout << "The average of those numbers is "
          << avgArray(userNums, SIZE) << endl;
    return 0;
}

// Write your function definition here
double avgArray(int num[], int size)
{
    double sum = 0;
    for (int i=0; i<size; i++)
    {
        sum += num[i];
    }
    return (sum/size);
}

```

Test: do (org-babel-tangle) first - this will create `avgArray.cpp` in the directory `../src/`

```

cd ../src
g++ avgArray.cpp -o avg
echo "1 2 3 4 5 6 7 8 9 10" | ./avg

```

Enter 10 numbers:

```

#1 1
#2 2
#3 3
#4 4
#5 5
#6 6
#7 7
#8 8
#9 9
#10 10

```

The average of those numbers is 5.5

35 Checkpoint (7.9 Arrays with Three or More Dimensions)

1. Define a two-dimensional array of integers named **grades**. It should have 30 rows and 10 columns.

```
int grades[30][10];
```

2. How many elements are in this array: `double sales[6][4];`

The 2-dimensional array has 24 elements in six rows x four columns.

3. Write a statement that assigns the value 56893.12 to the first column of the first row of the array `double sales[6][4];`:

```
double sales[6][4];
sales[0][0] = 56893.12;
```

4. Define a two-dimensional array named **settings** large enough to hold the table of data below. Then initialize the array with the values in the table.

12	24	32	21	42
14	67	87	65	90
19	1	24	12	8

Solution:

```
int settings[3][5] = {
    {12, 24, 32, 21, 42},
    {14, 67, 87, 65, 90},
    {19, 1, 24, 12, 8}
};
```

5. Fill in the table below so that it shows the contents of the following array:

```
int table[3][4] = {{2,3}, {7,9,2}, {1}};
```

Solution:

```
|---+---+---+---+---|
| 2 | 3 | 0 | 0 |   |
|---+---+---+---+---|
| 7 | 9 | 2 | 0 |   |
|---+---+---+---+---|
| 1 | 0 | 0 | 0 |   |
|---+---+---+---+---|
```

6. Write a function called **displayArray7**. The function should accept a two-dimensional array as an argument and display its contents on the screen. The function should work with any of the following arrays:

```
int hours[5][7];
int stamps[8][7];
int autos[12][7];
int cats[50][7];

#include <iostream>
using namespace std;

// Global constants
const int COLS = 7;
// Function prototype
void displayArray7(const int[][COLS], int);

int main()
{
    int hours[5][7] = {
        {1,2,3,4,5,6,7},
        {8,9,10,11,12,13,14},
        {15,16,17,18,19,20,21},
        {22,23,24,25,26,27,28},
        {29,30,31,32,33,34,35}
    };

    int stamps[8][7] = {
```

```

        {1,1,1,1,1,1,1},
        {2,2,2,2,2,2,2},
        {3,3,3,3,3,3,3},
        {4,4,4,4,4,4,4},
        {5,5,5,5,5,5,5},
        {6,6,6,6,6,6,6},
        {7,7,7,7,7,7,7},
        {8,8,8,8,8,8,8}
    };

    int autos[12][7] = {0}; // all zeros

    int cats[50][7] = {1}; // first element = 1, rest = 0

    cout << "hours:\n";
    displayArray7(hours,5);

    cout << "\nstamps:\n";
    displayArray7(stamps,8);

    cout << "\nautos:\n";
    displayArray7(autos,12);

    cout << "\ncats:\n";
    displayArray7(cats,50);

    return 0;
}
// Function definition
void displayArray7(const int a[][COLS], int rows)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < COLS; j++)
        {
            printf("%2d ", a[i][j]);
        }
        cout << endl;
    }
}

```

```
hours:
  1  2  3  4  5  6  7
  8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30 31 32 33 34 35
```

```
stamps:
  1  1  1  1  1  1  1
  2  2  2  2  2  2  2
  3  3  3  3  3  3  3
  4  4  4  4  4  4  4
  5  5  5  5  5  5  5
  6  6  6  6  6  6  6
  7  7  7  7  7  7  7
  8  8  8  8  8  8  8
```

```
autos:
  0  0  0  0  0  0  0
  0  0  0  0  0  0  0
  0  0  0  0  0  0  0
  0  0  0  0  0  0  0
  0  0  0  0  0  0  0
  0  0  0  0  0  0  0
  0  0  0  0  0  0  0
  0  0  0  0  0  0  0
  0  0  0  0  0  0  0
  0  0  0  0  0  0  0
  0  0  0  0  0  0  0
  0  0  0  0  0  0  0
```

```
cats:
  1  0  0  0  0  0  0
  0  0  0  0  0  0  0
  0  0  0  0  0  0  0
  0  0  0  0  0  0  0
  0  0  0  0  0  0  0
```

[illegible]

```

0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0

```

7. Write a program that displays the total number of books available at store. The store keeps books on 50 racks with 10 shelves each. Each shelf holds 25 books. Define a three-dimensional array large enough to represent the store's storage system.

Solution:

There are three constants: `RACK_SIZE = 50`, `SHELF_SIZE = 10`, and `BOOK_SIZE = 25` that define the three dimensions of the array. So there are at most $50 * 10 * 25 = 12500$ books in the store.

```

#include <iostream>
using namespace std;

int main()
{
    const int RACK_SIZE  = 50;
    const int SHELF_SIZE = 10;
    const int BOOK_SIZE  = 25;

    // Storage array:
    int storage[RACK_SIZE][SHELF_SIZE][BOOK_SIZE];

    cout << "Store can hold "
         << RACK_SIZE * SHELF_SIZE * BOOK_SIZE
         << " books.";

    return 0;
}

```

Store can hold 12500 books.

36 Case Study (7.10)

- Scenario:

The National Commerce Bank has hired you as a contract programmer. Your first assignment is to write a function that will be used by the bank's automated teller machines (ATMs) to validate a customer's personal identification number (PIN).

Your function will be incorporated into a larger program that asks the customer to input his PIN on the ATM's numeric keypad. PINs are seven-digit numbers. The program stores each digit in an element of an integer array. The program also retrieves a copy of the customer's actual PIN from a database. The PINs are also stored in the database as 7-element arrays. If these two numbers match, then the customer's identify is validated. Your function is to compare the two arrays and determine whether they contain the same numbers.

- Specifications:

1. **Parameters:** The function is to accept as arguments two integer arrays of seven elements each. The first argument will contain the number entered by the customer. The second argument will contain the number retrieved from the bank's database.
2. **Return:** The function should return a Boolean `true` value if the two arrays are identical. Otherwise, it should return `false`.

- Tasks: Get the driver to run. Save the function as a file `testPIN.cpp`, compile and link it to the driver program on the command line.

- Pseudocode for the function:

```
For each element in the first array
    Compare the element with the element in the second array
    that is in the corresponding position.
    If the two elements contain different values
        Return false.
    End If.
End For.
Return true.
```

- C++ Code for the function: (org-babel-tangle) to `../src/testPIN.cpp`.

```

//*****
// testPIN accepts two integer arrays. The arrays are compared.
// If they contain the same values, true is returned. If they
// contain different values, false is returned.
//*****
bool testPIN(const int custPIN[], const int databasePIN[], int size)
{
    for (int index = 0; index < size; index++)
    {
        if (custPIN[index] != databasePIN[index])
            return false; // We have discovered two different values.
    }
    return true; // If we make it this far, the values are the same.
}

```

- The Driver program: Because you've only been asked to write a function that performs the comparison between the customer's input and the PIN that was retrieved from the database, you will also need to design a **driver**, the program that supervises the database connection, calls the function etc. Tangles to ../src/testPINdrv.cpp.

```

// This program is a driver that tests a function comparing the
// contents of two integer arrays.

#include <iostream>
using namespace std;

// Function prototype
bool testPIN(const int [], const int [], int);

int main()
{
    const int NUM_DIGITS = 7; // Number of digits in a PIN.
    int pin1[NUM_DIGITS] = {2,4,1,8,7,9,0}; // Base set of values.
    int pin2[NUM_DIGITS] = {2,4,6,8,7,9,0}; // One element differs.
    int pin3[NUM_DIGITS] = {1,2,3,4,5,6,7}; // All element differ.

    if (testPIN(pin1, pin2, NUM_DIGITS))
        cout << "SUCCESS: pin1 and pin2 report to be the same.\n";
    else

```

```

        cout << "ERROR: pin1 and pin2 report to be different.\n";

    if (testPIN(pin1, pin3, NUM_DIGITS))
        cout << "SUCCESS: pin1 and pin3 report to be the same.\n";
    else
        cout << "ERROR: pin1 and pin3 report to be different.\n";

    if (testPIN(pin1, pin1, NUM_DIGITS))
        cout << "SUCCESS: pin1 and pin1 report to be the same.\n";
    else
        cout << "ERROR: pin1 and pin1 report to be different.\n";

    return 0;
}

```

- Testing the program: First the build (by hand, without make). We first build the object files for the function and for the driver. Then we link both object files to create the executable.

```

cd ../src
rm *.o testPIN
ls -lt testPIN*
g++ -c testPIN.cpp -o testPIN.o
g++ -c testPINdrv.cpp
g++ -o testPIN testPIN.o testPINdrv.o
ls -lt testPIN*

```

```

-rw-rw-r-- 1 aletheia aletheia 1010 Jul  8 14:14 testPINdrv.cpp
-rw-rw-r-- 1 aletheia aletheia  615 Jul  8 14:11 testPIN.cpp
-rwxrwxr-x 1 aletheia aletheia 16512 Oct 30 08:23 testPIN
-rw-rw-r-- 1 aletheia aletheia  3776 Oct 30 08:23 testPINdrv.o
-rw-rw-r-- 1 aletheia aletheia  1328 Oct 30 08:23 testPIN.o
-rw-rw-r-- 1 aletheia aletheia  1010 Jul  8 14:14 testPINdrv.cpp
-rw-rw-r-- 1 aletheia aletheia   615 Jul  8 14:11 testPIN.cpp

```

- Run the executable:

```

cd ../src
./testPIN

```

```
ERROR: pin1 and pin2 report to be different.
ERROR: pin1 and pin3 report to be different.
SUCCESS: pin1 and pin1 report to be the same.
```

37 TODO 7.11 Introduction to the STL vector (page 443)

- The `vector` data type of the Standard Template Library (STL) is similar to an array:
 1. Holds a sequence of values or elements (of one type).
 2. Stores its elements in contiguous memory locations.
 3. Can be accessed using the subscript operator `[]` (only on existing `vector` elements).
- Some differences:
 1. You don't have to declare the number of elements that the `vector` will have.
 2. If you add a value to a `vector` that's already full, the `vector` will automatically increase its size to accommodate the new value.
 3. A `vector` can report the number of elements it contains.
 4. To use `vector` you need `using namespace std;` in your program (or otherwise declare `std::vector`).
 5. You need to `#include <vector>`.
- What the doc says:

You need to `#include <vector>` - check out `cppreference`: "`std::vector` is a sequence container that encapsulates dynamic size arrays."

Further down the page, you see some notes on the algorithmic complexity (aka efficiency) of common `vector` operations:

 1. Random access (pick any element): $O(1)$
 2. Insertion or removal of elements at the end: $O(1)$
 3. Insertion or removal of elements anywhere: $O(n)$
- Important member functions: `swap`, `clear`, `empty`, `push_back`, `insert`, `resize`, `pop_back`, `at`.

38 Declaring vectors

- Example: We define `numbers` as a vector of integers `vector<int>`.

```
vector<int> numbers;  
vector<int> numbers2(10); // Declaring a starting size is optional.  
vector<int> numbers3(10,2); // Initializes 10 numbers to the value 2.  
vector<int> numbers4(numbers3); // Copies vector numbers3 into vector numbers4  
vector<int> num {10,20,30,40}; // Initializes vector with value set.
```

- Overview of vector definitions:

Definition Format	Description
<code>vector<float> amounts;</code>	Defines <code>amounts</code> as an empty vector of floats.
<code>vector<string> names;</code>	Defines <code>names</code> as an empty vector of string objects.
<code>vector<int> scores(15);</code>	Defines <code>scores</code> as a vector of 15 ints.
<code>vector<char> letters(25, 'A');</code>	Defines <code>letters</code> as a vector of 25 characters. Each element is initialized with 'A'.
<code>vector<double> values2(values1);</code>	Defines <code>values2</code> as a vector of doubles. All the elements of <code>values1</code> , which is also a vector of doubles, are copied to <code>value2</code> .

39 Storing values in a vector (payroll program revisited)

- We come back to the payroll program. This time, we store the `hours` and the `payRate` not in parallel arrays but in two vectors.

```
// Compute the grossPay of NUM_EMPLOYEES who work certain hours and  
// receive a certain payRate.  
// Input: integer hours and double payRate for each employee  
// Sample output: Hours worked/payRate/grossPay received for each employee  
// Author: Marcus Birkenkrahe (based on Gaddis, p.444)  
#include <iostream>  
#include <iomanip>  
#include <vector>  
using namespace std;
```

```

int main()
{
    // Declare variables.
    const int NUM_EMPLOYEES = 5;
    vector<int> hours(NUM_EMPLOYEES);
    vector<double> payRate(NUM_EMPLOYEES);
    int index; // Loop index
    double grossPay; // Hours x pay rate

    // Input the data.
    cout << "Enter the hours worked by " << NUM_EMPLOYEES
         << " employees and their hourly rates.\n";

    for (index = 0; index < NUM_EMPLOYEES; index++)
    {
        cout << "Hours worked by employee #" << (index + 1)
             << ": ";
        cin >> hours[index];
        cout << hours[index] << endl;
        cout << "Hourly pay rate for employee #" << (index + 1)
             << ": ";
        cin >> payRate[index];
        cout << payRate[index] << endl;
    }

    // Display each employee's gross pay (hours x pay rate)
    cout << "\nHere is the gross pay for each employee:\n";
    cout << fixed << showpoint << setprecision(2);
    for (index = 0; index < NUM_EMPLOYEES; index++)
    {
        double grossPay = hours[index] * payRate[index];
        cout << "Employee #" << (index + 1)
             << ": $" << grossPay << endl;
    }

    return 0;
}

```

- Compile and run with sample input:

```
cd ../src
g++ payroll2.cpp -o pay
echo "10 9.75 15 8.62 20 10.50 40 18.75 40 15.65" | ./pay
```

Enter the hours worked by 5 employees and their hourly rates.

```
Hours worked by employee #1: 10
Hourly pay rate for employee #1: 9.75
Hours worked by employee #2: 15
Hourly pay rate for employee #2: 8.62
Hours worked by employee #3: 20
Hourly pay rate for employee #3: 10.5
Hours worked by employee #4: 40
Hourly pay rate for employee #4: 18.75
Hours worked by employee #5: 40
Hourly pay rate for employee #5: 15.65
```

Here is the gross pay for each employee:

```
Employee #1: $97.50
Employee #2: $129.30
Employee #3: $210.00
Employee #4: $750.00
Employee #5: $626.00
```

40 Using the range-based for loop with a vector (from C++11)

- Example 1:

```
// Display vector elements using a range-base for loop
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    // Define and initialize a vector.
    vector<int> numbers {10, 20, 30, 40, 50};
    // Display the vector elements.
    for (int val : numbers)
```

```

        cout << val << " ";

    return 0;
}

```

10 20 30 40 50

- Example 2:

```

// Using a range-based for loop, fill a vector with user input and
// display the elements
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    // Define a vector.
    vector<int> numbers (5);
    // Get values for the vector elements.
    for (int &val : numbers)
    {
        cout << "Enter an integer value: ";
        cin >> val; // get value
    }
    // Display the vector elements.
    cout << "Here are the values you entered:\n";
    for (int val : numbers)
        cout << val << endl;
    return 0;
}

```

```

Enter an integer value: Enter an integer value: Enter an integer value: Enter an i
0
0
0
0
0
0

```

- Input file:

```
cd ../data
echo "10 20 30 40 50" > inputFile
cat inputFile
```

```
10 20 30 40 50
```

41 Using the `push_back` member function (payroll pgm modified)

- You use `push_back` to store a value in a `vector` without starting size, or to append it at the end of a `vector` that is already full.
- Example:

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> numbers;           // empty vector, no starting size
    numbers.push_back(10);        // add first element
    numbers.push_back(20);        // add second element
    numbers.push_back(30);        // add third element

    for(int n : numbers)
        cout << n << " ";

    return 0;
}
```

```
10 20 30
```

- Modify ?? : Change the `vector` definitions to vectors without starting size, and use `push_back` to fill the vectors.
- Solution:

```

// payroll3.cpp stores, in two arrays, the hours worked
// by the employees and their hourly pay rates, and computes
// and displays the gross pay for each employee.
#include <iostream>
#include <iomanip>
#include <vector>
using namespace std;

int main()
{
    // Declare variables.
    vector<int> hours;
    vector<double> payRate;
    int numEmployees;

    // Get the number of employees.
    cout << "How many employees do you have?\n";
    cin >> numEmployees;

    // Input the data.
    cout << "Enter the hours worked by " << numEmployees
         << " employees and their hourly rates.\n";

    for (int index = 0; index < numEmployees; index++)
    {
        int tempHours;
        double tempRate;
        cin >> tempHours; // get hours for employee index + 1
        hours.push_back(tempHours);
        cin >> tempRate; // get pay rate for employee index + 1
        payRate.push_back(tempRate);
    }

    // Display each employee's gross pay (hours x pay rate)
    cout << "\nHere is the gross pay for each employee:\n";
    cout << fixed << showpoint << setprecision(2);
    for (int index = 0; index < numEmployees; index++)
    {
        double grossPay = hours[index] * payRate[index];
        cout << "Employee #" << (index + 1)

```

```

        << ": $" << grossPay << endl;
    }

    return 0;
}

```

- Compile and run with sample input:

```

cd ../src
g++ payroll3.cpp -o pay
echo "5 10 9.75 15 8.62 20 10.50 40 18.75 40 15.65" | ./pay

```

How many employees do you have?
Enter the hours worked by 5 employees and their hourly rates.

Here is the gross pay for each employee:
Employee #1: \$97.50
Employee #2: \$129.30
Employee #3: \$210.00
Employee #4: \$750.00
Employee #5: \$626.00

- Why can't you use `.push_back` directly in a `cout << stream`?

This method is **mutating** - it modifies the state of the **vector** container and returns **void** (see here). The `<<` operator in a stream expression requires its right-value operand to be an object and not **void**.

42 Determining the size of a vector

- Example 1: This code uses the **assert** macro to check the **size** of the **vector**. There is no output because **assert** succeeds (cppreference).

```

#include <iostream>
#include <cassert>
#include <vector>

int main()

```

```

{
    std::vector<int> nums;
    assert(nums.size() == 0);
    nums = {1,2,3,4};
    assert(nums.size() == 3);
    return 0;
}

```

- Example 2: Demonstrate the `vector` method `size`. Notice that the function arguments does not need to include the size since `vector` is dynamically allocated.

```

#include <iostream>
#include <vector>
using namespace std;

// Function prototype
void showValues(vector<int>);

int main()
{
    // Define a vector.
    vector<int> values;

    // Put a series of numbers in the vector.
    for (int count = 0; count < 7; count++)
        values.push_back(count * 2);

    // Display the numbers.
    showValues(values);
    return 0;
}

//*****
// showValues accepts an integer vector as its argument.    *
// The value of each of the vector's elements is displayed. *
//*****
void showValues(const vector<int> v)
{
    for (int count = 0; count < v.size(); count++)

```

```
        cout << v[count] << " ";
    }
}
```

```
0 2 4 6 8 10 12
```

43 Removing elements from a vector with `pop.back`

- `.pop_back()` removes the last element from the vector.
- Example:

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> values;

    // Store values in the vector.
    values.push_back(1);
    values.push_back(2);
    values.push_back(3);
    cout << "The size of values is " << values.size() << endl;

    // Remove a value from the vector.
    cout << "Popping a value from the vector ...\n";
    values.pop_back();
    cout << "The size of values is now " << values.size() << endl;

    // Remove a value from the vector.
    cout << "Popping a value from the vector ...\n";
    values.pop_back();
    cout << "The size of values is now " << values.size() << endl;

    // Remove a value from the vector.
    cout << "Popping a value from the vector ...\n";
    values.pop_back();
    cout << "The size of values is now " << values.size() << endl;
}
```

```

    return 0;
}

```

```

The size of values is 3
Popping a value from the vector ...
The size of values is now 2
Popping a value from the vector ...
The size of values is now 1
Popping a value from the vector ...
The size of values is now 0

```

- Removing another value from an already empty `vector` causes undefined behavior and nonsense output.

44 Clearing a vector

- To clear the contents of a `vector`, use the `.clear()` member function.
- Example:

```

#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> values(100);

    cout << "The 'values' vector has "
          << values.size() << " elements.\n";

    values.clear();

    cout << "Now the 'values' vector has "
          << values.size() << " elements.\n";

    return 0;
}

```

The 'values' vector has 100 elements.
Now the 'values' vector has 0 elements.

45 Detecting an empty vector

- To determine if a `vector` has no content (that is no memory reserved for it), use the `.empty` member function.
- Example: To demonstrate the use of the `empty` function, we store values in a `vector`, and then determine the average of these values. If the vector is not empty, there are no values to average and we set the average to zero.

```
#include <iostream>
#include <vector>
#include <iomanip>
using namespace std;

// Function prototype
double avgVector(vector<int>);

int main()
{
    vector<int> values;
    int numValues;
    double average;

    // Get the number of values to average.
    cout << "How many values do you wish to average? ";
    cin >> numValues;

    // Get the values and store them in the vector.
    for (int count = 0; count < numValues; count++)
    {
        int tempValue;
        cout << "Enter an integer value: ";
        cin >> tempValue;
        values.push_back(tempValue);
    }
```

```

        // Get the average of the values and display it.
        average = avgVector(values);
        cout << fixed << showpoint << setprecision(2);
        cout << "\nAverage: " << average << endl;

    return 0;
}

//*****
// avgVector accepts an integer vector as its argument *
// If the vector contains values, the function returns *
// the average of those values as a double. Otherwise, *
// an error message is displayed and 0.0 is returned. *
//*****
double avgVector(vector<int> vect)
{
    double total = 0;
    double avg;
    if (vect.empty())
    {
        cout << "\nVector is empty - nothing to average.\n";
        avg = 0.0;
    }
    else
    {
        for (int count = 0; count < vect.size(); count++)
            total += vect[count];
        avg = total / vect.size();
    }
    return avg;
}

```

```

How many values do you wish to average? Enter an integer value: Enter an integer v
Average: 44.00

```

- InputFile:

```

cd ../data
echo "5 12 18 3 7 9" > inputFile

```

```
#echo "0" > inputFile
cat inputFile
```

```
5 12 18 3 7 9
```

46 PRACTICE Summary of vector member functions

- `at(element)` returns the value of the element located at `element` in the vector:

```
vector<int> vect(100,10);
cout << vect.at(5);
```

```
10
```

- `clear()` clears a vector of all its elements:

```
vector<int> vect(100,10);
cout << vect.size() << endl;
vect.clear();
cout << vect.size();
```

```
100
0
```

- `empty()` returns `true` if the vector is empty, and `false` otherwise.

```
vector<int> vect;
cout << boolalpha << vect.empty() << endl;
vector<int> vect2(100);
cout << boolalpha << vect2.empty();
```

```
true
false
```

- `pop_back` removes the last element from the `vector` thus reducing its size by one.

```
vector<int> vect(100);
cout << vect.size() << endl;
vect.pop_back();
cout << vect.size() << endl;
```

```
100
99
```

- `push_back(value)` stores `value` in the last element of the `vector`. If the `vector` is full or empty, a new element is created.

```
vector<int> vect;                // vector is empty
cout << vect.size() << endl;
vect.push_back(10);              // vector is full
cout << vect.size() << endl;
vect.push_back(100);
cout << vect.size() << endl;
```

```
0
1
2
```

- `resize(num, value)` resizes a `vector` by `num` elements and initializes each of the new elements with `value`. If `num` is the same as the current size, it does nothing.

```
vector<int> vect = {1,2,3,4,5};
cout << vect.size() << endl;
vect.resize(10,100);
cout << vect.size() << endl;
cout << vect[9];
```

```
5
10
100
```

- `swap(vector2)` swaps the content of the `vector` with the contents of `vector2`.

```
vector<int> vect1 = {10,20,30};
vector<int> vect2 = {40,50,60};
vect1.swap(vect2);
for (int val : vect1)
    cout << val << " ";
cout << '\n';
for (int val : vect2)
    cout << val << " ";
```

```
40 50 60
10 20 30
```

47 Checkpoint (7.11 vectors)

1. What header file must you `#include` in order to define `vector` objects?
If you don't, what error message do you get?

To define `vector` objects you need to `#include <vector>`.

```
#include <iostream>
// 'std::vector' is defined in header '<vector>'; did you forget to
// '#include <vector>'?
using namespace std;

int main()
{
    vector<int> vect; // 'vector' was not declared in this scope
    return 0;
}
```

2. Write a definition statement for a `vector` named `frogs`. `frogs` should be an empty `vector` of integer values. Show that.

```
vector<int> frogs;
if (frogs.empty()) cout << "frogs is empty.";
```

frogs is empty.

3. Write a definition statement for a **vector** named **lizards**. **lizards** should be an **vector** of 20 floating-point values. Show that.

```
vector<float> lizards(20);  
cout << lizards.size();
```

20

4. Write a definition statement for a **vector** named **toads**. **toads** should be a **vector** of 100 characters, with each element initialized to 'Z'. Show that using a range-based loop.

```
vector<char> toads(100,'Z');
for (char toad : toads)
    cout << toad << " ";
cout << '\n' << toads.size();
```

[illegible]

5. `gators` is an empty **vector** of integer values. Write a statement that stores the value 27 in `gators`. Show that.

```
vector<int> gators;
if (gators.empty())
    gators.push_back(27);
for (int gator : gators)
    cout << gator << " ";
cout << '\n' << gators.size();
```

27

1

6. `snakes` is a `vector` of double precision floating-point values, with 10 elements. Write a statement that stores the value `12.897` in element 4 of the `snakes` vector, print it and then remove it.

```
vector<double> snakes(10);
snakes.push_back(12.897);
for (double snake : snakes)
    cout << snake << " ";
snakes.pop_back();
cout << '\n' << snakes.size() << endl;
for (double snake : snakes)
    cout << snake << " ";
```

```
0 0 0 0 0 0 0 0 0 0 12.897
10
0 0 0 0 0 0 0 0 0 0
```

48 Glossary

Table 1: Pointer Glossary

Aspect	Pointer Example / Description
Purpose	Holds a memory address, can point to vars or be nullptr
Define	<code>int *p; p = &i;</code>
Access	<code>*p</code> dereferences; <code>p</code> gives the address
Copy	<code>q = p;</code> now both point to same object
Extend	Use pointer arithmetic or point to other pointers
Delete	Set to nullptr; use delete if allocated w/ new
Risks	Uninitialized/dangling pointers cause UB
Use cases	Linked lists, trees, function args, dynamic memory

Table 2: Array Glossary	
Aspect	Array Example / Description
Purpose	Store multiple values of the same type
Define	<code>int a[3] = {1, 2, 3};</code>
Access	<code>a[0]</code> , <code>a[i]</code> , pointer-style: <code>*(a+i)</code>
Copy	Manual loop: <code>for (i) b[i] = a[i];</code>
Extend	Allocate new array w/ <code>new</code> ; copy + add elements
Delete	Use <code>delete[]</code> for dynamic arrays
Storage	Contiguous memory layout
Subscripts	Always 0 to n-1

Table 3: Emacs Org-mode Glossary	
Aspect	Description
<code>src block</code>	<code>#+begin_src</code> C++ starts a code block
<code>:main</code>	<code>:main</code> yes includes a <code>main()</code> wrapper
<code>:includes</code>	Headers to include, e.g. <code><iostream></code>
<code>:results</code>	<code>:results</code> output prints program output
<code>:exports</code>	<code>:exports</code> both shows code and result
<code>:cmdline</code>	Simulates keyboard input with <code><</code> input
<code>:tangle</code>	Writes code block to file, e.g. <code>cin.cpp</code>
<code>C-c C-v t</code>	Runs <code>org-babel-tangle</code> to export code to file